

## Laboratoire d'Assembleur semestre printemps 2026

### Laboratoire n°6

Temps à disposition : 4 périodes (deux séances de laboratoire)

### Objectifs de laboratoire

L'objectif de ce laboratoire est d'apprendre à utiliser les instructions de branchement, de se familiariser avec la pile ainsi que de se familiariser avec l'ABI ARM.

### Validation du laboratoire

Un script de rendu vous est fourni afin de packager les fichiers sources modifiés sans inclure tous les fichiers générés (code objet, exécutables, etc.). Ce script s'appelle rendu.sh et sera fourni pour tous les laboratoires.

```
reds@reds: /asm/asm_lab0X$ ./rendu.sh
```

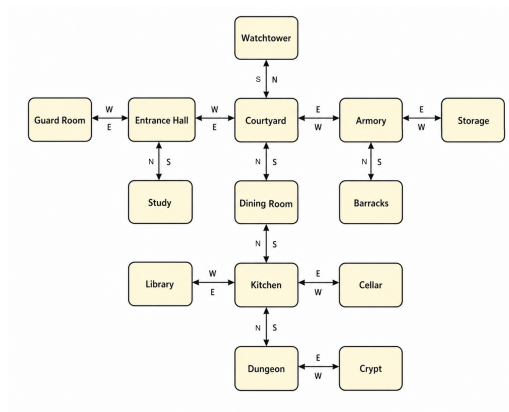
Cette commande va générer un fichier asm\_lab0X\_rendu.tar.gz (où X est le numéro du labo) et c'est cette archive qui sera à remettre sur Cyberlearn. Il est demandé de toujours préparer l'archive du laboratoire grâce au script de rendu fourni.

### Récupération des sources environnement Eclipse

```
$ retrieve_lab asm26 lab06
```

## Étape 1 - Parcours de labyrinthe (x86)

Dans cette étape, il s'agira d'écrire un algorithme permettant d'explorer un labyrinthe représenté sous forme de graphe à l'aide d'une pile.



Un labyrinthe est ici modélisé comme un ensemble de salles reliées entre elles. Chaque salle possède jusqu'à 4 voisines :

1. Nord
2. Est
3. Sud
4. Ouest

Certaines directions peuvent être absentes (valeur nulle).

Dans cette étape, il s'agira d'écrire un algorithme qui permet de parcourir toutes les salles du labyrinthe en utilisant un parcours en profondeur (DFS) avec une pile explicite.

Chaque salle contient un nom (chaîne de caractères). Lorsqu'une salle est visitée pour la première fois, son nom doit être affiché. L'ordre des noms affichés permet de vérifier que le parcours du labyrinthe respecte bien l'ordre d'exploration imposé.

En mémoire, une salle utilise la représentation suivante :

```
Room {
    Data *data;
    Room *north;
    Room *west;
    Room *east;
    Room *south;
    Room *west;
    int visited;
};
```

| Mémoire (32-bits) | Offset |
|-------------------|--------|
| visited           | + 0x14 |
| Room * west       | + 0x10 |
| Room * south      | + 0x0c |
| Room * east       | + 0x08 |
| Room * north      | + 0x04 |
| Data * ptr        | + 0x00 |

Le champ `visited` permet de marquer une salle comme déjà visitée afin d'éviter les boucles infinies dans le cas de cycles.

L'ordre logique d'exploration des directions est le suivant :

**Nord → Est → Sud → Ouest**

**Attention :** étant donné que la pile est de type LIFO (Last In First Out), les voisins devront être empilés dans l'ordre inverse afin de respecter cet ordre d'exploration.

- a) Écrire l'algorithme qui permet de parcourir le labyrinthe et afficher les données (data) associées à chaque salle dans le fichier `x86/binary_tree/maze_solver_asm.S`.

#### Information

- Un pointeur sur la salle de départ du labyrinthe est passé en paramètre.
- Utiliser la pile pour stocker les adresses des salles qui doivent encore être explorées.
- La macro **PUT\_S** permet d'afficher le nom d'une salle et prend en argument un pointeur sur une chaîne de caractères.
- Une salle ne doit être affichée qu'une seule fois.
- Si une salle est nulle, elle ne doit pas être affichée.
- Lorsqu'une direction n'existe pas, le pointeur correspondant vaut 0.

#### Avertissement

À la fin de l'algorithme, la pile doit être dans le même état (contenu et position du pointeur) qu'avant l'appel de la fonction.

À titre d'exemple, un labyrinthe correctement parcouru pourrait produire la sortie suivante :

Entrance Corridor Throne\_room Library Armory

Cette sortie correspond à l'ordre dans lequel les salles sont visitées par le parcours DFS, en respectant l'ordre d'exploration :

**Nord → Est → Sud → Ouest**

Si l'ordre d'exploration n'est pas respecté, si une salle est visitée plusieurs fois, ou si la pile n'est pas correctement utilisée, l'ordre des salles affichées sera incorrect.

Contrairement à un arbre binaire de recherche, l'ordre du parcours n'est pas déterminé par une propriété de tri. Il est déterminé par la structure du labyrinthe et par l'ordre d'exploration défini. Les labyrinthes fournis sont construits de manière à produire une sortie attendue déterministe.

## Étape 2 - ARM ABI AAPCS – Backtrace

Dans cette étape nous allons écrire une fonction qui permet d'afficher une « backtrace »<sup>1</sup> aussi appelée « stack trace ». C'est la trace hiérarchique des appels de fonctions effectués dans un programme à un endroit donné. En général la « backtrace » est utilisée lors de debug, avec GDB par exemple et permet de savoir quelle est la séquence d'appels de fonction jusqu'à l'endroit donné.

Le but de cet exercice sera d'afficher les adresses de retour vers les fonctions appelantes. Par exemple si « main() » appelle « foo() » qui appelle « bar() » et qu'on aimerait afficher la « backtrace » dans « bar() » on aurait des adresses dans « bar() – foo() – main() ». Afin de réaliser ceci, nous allons parcourir la pile « stack ».

|            |    |                                                           |
|------------|----|-----------------------------------------------------------|
| [fp, #12]  |    | - 3 <sup>ème</sup> argument supplémentaire de la fonction |
| [fp, #8]   |    | - 2 <sup>ème</sup> argument supplémentaire de la fonction |
| [fp, #4]   |    | - 1 <sup>er</sup> argument supplémentaire de la fonction  |
| fp         | lr | - adresse de retour                                       |
| [fp, #-4]  | fp | - valeur de <b>fp</b> préservée (ancienne)                |
| [fp, #-8]  |    | - 1 <sup>ère</sup> variable locale                        |
| [fp, #-12] |    | - 2 <sup>ème</sup> variable locale                        |
| [fp, #-16] |    | - 3 <sup>ème</sup> variable locale                        |

Selon l'ABI ARM AAPCS chaque fonction appelée va enregistrer l'adresse de retour (vers la fonction appelante) ainsi que le « frame pointer » (de l'appelant) sur la pile.

Pour l'exercice nous avons instrumenté la pile afin qu'avant que « main() » soit appelé, les valeurs -1, -1 (0xFFFF'FFFF, 0xFFFF'FFFF) soient écrits sur la pile avant d'appeler « main() ». Ceci nous permettra de trouver la base de la pile pour notre application. Ceci est réalisé dans le fichier « crt0.S » juste avant d'appeler « main() ».

Il s'agira ici d'écrire le code assembleur requis pour afficher les adresses de retour (« lr ») enregistrés sur la pile. Les valeurs « fp » sauvegardées nous permettront de localiser les piles des appelants. Jusqu'à ce que les valeurs de -1, -1 (pour « fp » et « lr ») soient trouvées (c'est le sommet de la chaîne d'appels).

- Compléter la fonction « show\_backtrace() » dans le fichier « arm/backtrace/backtrace\_asm.S » qui va afficher l'un après l'autre la valeur du « lr » précédent jusqu'au sommet de la pile.
- Tester dans QEMU avec l'application « backtrace.bin ». Cette application va appeler des fonctions imbriquées dans un ordre aléatoire, jusqu'à appeler « show\_backtrace() ».
- Le log de la sortie de QEMU (« ./st.log ») est enregistré dans le fichier « st.log ». Grâce au script « ./addr2fun.sh » la sortie de ce fichier est filtrée et les adresses sont traduites vers les noms de fonctions correspondantes (voir exemple ci-dessous). Ceci permet de valider que les adresses correspondent bien aux fonctions appelées (l'ordre sera en miroir des appels puisqu'on affiche la valeur de lr vers l'appelant depuis la fonction la plus profonde).

```
~/asm22_labo6$ less st.log
```

```
----
Function main called
Function function_D called

Backtrace :
----
Previous link register : 0x820003a0
Previous link register : 0x820004cc
Previous link register : 0x82000020
----
```

Hit any key to exit ...

```
~/asm22_labo6$ ./addr2fun.sh
```

```
----
```

```
Function main called
```

```
Function function_D called
```

```
----
```

```
function_D at backtrace.c:87 (discriminator 1) <= 0x820003a0
```

```
main at backtrace.c:100 <= 0x820004cc
```

```
\$a at crt0.S:23 <= 0x82000020
```

```
----
```

### Information

Utiliser la fonction « printf » afin d’afficher les adresses avec la « format string » fournie dans le fichier assembleur.

### Avertissement

- Attention à ne pas corrompre (modifier) la pile.
- Attention à ne pas écraser des registres non « scratch », les sauvegarder sur la pile et les restaurer si vous en faites usage.
- Attention la fonction « printf » peut modifier les registres « scratch ».